Apache Airflow DAG: S3 File to Database Pipeline

Problem Statement

Create an Apache Airflow DAG that:

- Reads a data file uploaded to AWS S3
- Loads the data into a database
- Uses Airflow Connections for:
  - AWS S3 connection
  - Database connection
- Uses Airflow Variables for:
  - S3 bucket
  - File path
  - Table name
- Runs daily at **12:10 PM**
- Utilize Blob Storage to SQL operators if available

Solution Architecture

AWS S3 ---> Airflow DAG ---> Database (PostgreSQL/MySQL)

Prerequisites

Install required providers:

```
pip install apache-airflow-providers-amazon
pip install apache-airflow-providers-postgres
pip install pandas
pip install sqlalchemy
pip install psycopg2-binary
```

Step 1: Create Airflow Connections

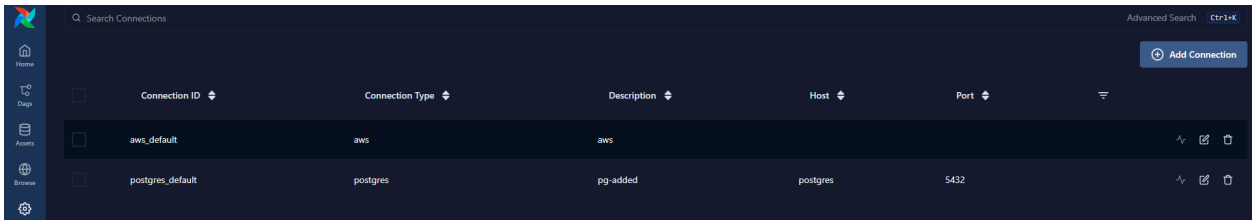
AWS Connection

Go to:

Airflow UI → Admin → Connections

Create connection:

Field	Value
Connection Id	aws_default
Connection Type	Amazon Web Services
AWS Access Key	YOUR_ACCESS_KEY
AWS Secret Key	YOUR_SECRET_KEY



PostgreSQL Connection

Field	Value
Connection Id	postgres_default
Connection Type	Postgres
Host	localhost
Schema	airflow_db
Login	postgres
Password	postgres
Port	5432

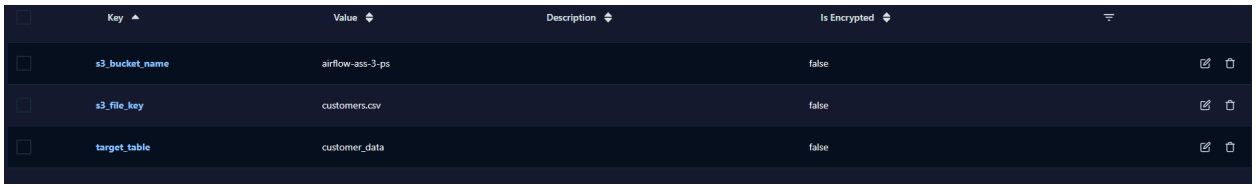
Step 2: Create Airflow Variables

Go to:

Airflow UI → Admin → Variables

Create variables:

Key	Value
s3_bucket_name	my-data-bucket
s3_file_key	input/customer_data.csv
target_table	customer_data



Key	Value	Description	Is Encrypted
s3_bucket_name	airflow-ass-3-ps		false
s3_file_key	customers.csv		false
target_table	customer_data		false

Step 3: Create Database Table

Example PostgreSQL table:

```
CREATE TABLE customer_data (
  id INT,
  name VARCHAR(100),
  city VARCHAR(100),
  salary INT
);
```

Step 4: DAG Implementation

Create file:

dags/s3_to_postgres_pipeline.py

Complete DAG Code

```
from airflow import DAG
from airflow.operators.python import PythonOperator
from airflow.models import Variable
from airflow.providers.amazon.aws.hooks.s3 import S3Hook
from airflow.providers.postgres.hooks.postgres import PostgresHook

from datetime import datetime
import pandas as pd
from io import StringIO


# Fetch Airflow Variables
S3_BUCKET = Variable.get("s3_bucket_name")
S3_FILE_KEY = Variable.get("s3_file_key")
TARGET_TABLE = Variable.get("target_table")

# Connections
AWS_CONN_ID = "aws_default"
POSTGRES_CONN_ID = "postgres_default"


default_args = {
  "owner": "airflow",
  "start_date": datetime(2026, 5, 1),
  "retries": 1
}


def load_s3_file_to_postgres():

  # Initialize S3 Hook
  s3_hook = S3Hook(aws_conn_id=AWS_CONN_ID)

  # Read file from S3
  file_obj = s3_hook.get_key(
      key=S3_FILE_KEY,
      bucket_name=S3_BUCKET
  )

  file_content = file_obj.get()["Body"].read().decode("utf-8")

  # Convert CSV to Pandas DataFrame
```

```
    df = pd.read_csv(StringIO(file_content))

    # Initialize Postgres Hook
    postgres_hook = PostgresHook(
        postgres_conn_id=POSTGRES_CONN_ID
    )

    # SQLAlchemy Engine
    engine = postgres_hook.get_sqlalchemy_engine()

    # Load data into PostgreSQL
    df.to_sql(
        TARGET_TABLE,
        engine,
        if_exists='append',
        index=False
    )

    print("Data successfully loaded from S3 to PostgreSQL")


with DAG(
    dag_id="s3_to_postgres_pipeline",
    default_args=default_args,
    schedule="10 12 * * *",   # Runs daily at 12:10 PM
    catchup=False,
    tags=["s3", "postgres", "etl"]
) as dag:

    load_data_task = PythonOperator(
        task_id="load_s3_to_postgres",
        python_callable=load_s3_file_to_postgres
    )

    Load_data_task
```

```
airflow=# SELECT * FROM customer_data;
 customer_id | name
-------------+-------
           1 | John
           2 | Alice
           3 | Bob
           1 | John
           2 | Alice
           3 | Bob
(6 rows)
```

Cron Explanation

10 12 * * *

Meaning:

Value	Meaning
10	Minute
12	Hour
*	Every day
*	Every month
*	Every weekday

So DAG runs at: 12:10 PM Daily

Project Structure

```
airflow/
│
├── dags/
│   └── s3_to_postgres_pipeline.py
│
├── logs/
│
├── plugins/
```

How This DAG Works

Step-by-Step Flow

Step 1

Scheduler triggers DAG at:

12:10 PM daily

Step 2

DAG fetches variables:

Variable.get()

Used for:

- bucket name
- file path
- table name

Step 3

Airflow uses:

S3Hook to connect AWS S3.

Step 4

CSV file is downloaded from S3.

Step 5

Pandas reads CSV into DataFrame.

pd.read_csv()

Step 6

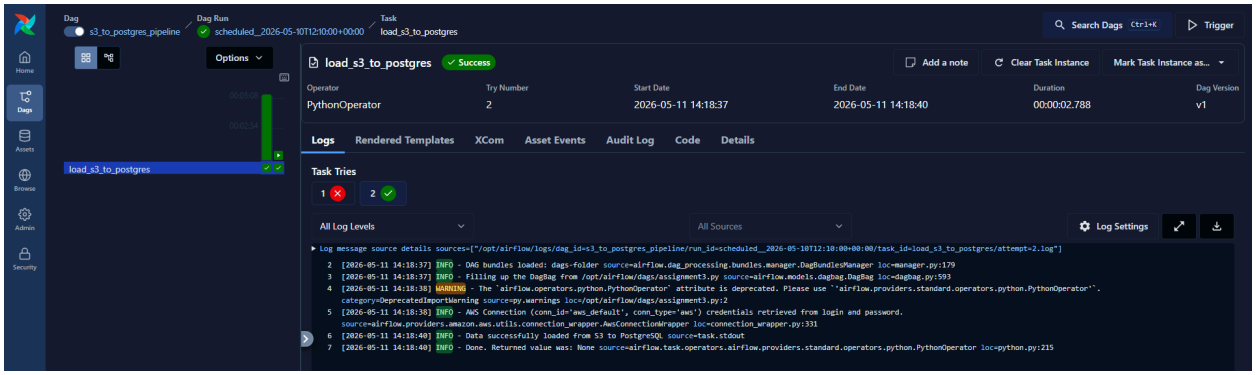
Airflow uses:

PostgresHook to connect PostgreSQL.

Step 7

Data is inserted into database table using:

df.to_sql()



Important Airflow Concepts Used

Concept	Usage
DAG	Workflow definition
PythonOperator	Execute Python code
S3Hook	Connect AWS S3
PostgresHook	Connect PostgreSQL
Variable	Store constant values
Connection	Store credentials
Cron Schedule	Run daily

Production Enhancements

Add File Sensor

Use:

S3KeySensor

to wait for file arrival.

Add Data Validation

Validate:

- Null values
- Duplicate records
- Schema mismatch

Add Logging

Use Airflow logs for debugging.

Add Retry Mechanism

retries=3
retry_delay=timedelta(minutes=5)

Archive Processed Files

Move processed files to:

s3://bucket/archive/

Final Output

This pipeline:

✔ Reads CSV from AWS S3
✔ Uses Airflow Variables
✔ Uses Airflow Connections
✔ Loads data into PostgreSQL
✔ Runs daily at 12:10 PM
✔ Follows production ETL design principles